



EKS Session

Summary 30-04-2023

- Manage the route rules in an EKS (Elastic Kubernetes Service) cluster using Kubernetes (k8s) rather than AWS Cloud by deploying **Custom Resource Definitions (CRDs)**.
- CRDs allow you to define your own custom resources in Kubernetes. **Ingress CRDs allow you to define custom routing rules.**
- In Kubernetes, you can use the Ingress resource to manage route rules. An Ingress is a Kubernetes resource that provides a way to manage external access to services in a cluster. You can define rules in an Ingress that determine how traffic should be routed to different services based on the requested path or hostname.

```
C:\Users\Vimal Daga>kubect1 get crd
NAME                                     CREATED AT
eniconfigs.crd.k8s.amazonaws.com       2023-04-29T05:45:28Z
ingressclassparams.elbv2.k8s.aws        2023-04-29T06:53:30Z
securitygrouppolicies.vpcresources.k8s 2023-04-29T05:45:31Z
targetgroupbindings.elbv2.k8s.aws       2023-04-29T06:53:30Z
```

- **Ingress rules Example:** Launch 2 services with deployments.
 - Launch deployment for mymail.

[EKS]

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: mail
spec:
  replicas: 3
  selector:
    matchLabels:
      app: mymail
    matchExpressions:
      - { key: app, operator: In, values: [mymail] }
  template:
    metadata:
      name: mymail-pod
      labels:
        app: mymail
    spec:
      containers:
        - name: myweb-con
          image: vimal13/apache-webserver-php
```

```
C:\Users\Vimal Daga\Desktop\kube_code>notepad deploy.yml

C:\Users\Vimal Daga\Desktop\kube_code>kubectl apply -f deploy.yml
deployment.apps/mail created

C:\Users\Vimal Daga\Desktop\kube_code>kubectl ge tdeploy
error: unknown command "ge" for "kubectl"

Did you mean this?
  set
  get
  cp

C:\Users\Vimal Daga\Desktop\kube_code>kubectl get deploy
NAME          READY   UP-TO-DATE   AVAILABLE   AGE
mail          3/3     3            3           8s
myweb-deploy  3/3     3            3           22h
```

- Launch service for mymail deployment.

```
apiVersion: v1
kind: Service
metadata:
  name: mymail-service
  labels:
    app: mymail-service
spec:
  type: NodePort
  selector:
    app: mymail
  ports:
    - port: 80
      targetPort: 80
      nodePort: 31001
```

```
C:\Users\Vimal Daga\Desktop\kube_code>notepad service.yml

C:\Users\Vimal Daga\Desktop\kube_code>kubectl apply -f service.yml
service/mymail-service created

C:\Users\Vimal Daga\Desktop\kube_code>kubectl get svc
NAME                TYPE        CLUSTER-IP    EXTERNAL-IP  PORT(S)          AGE
kubernetes          ClusterIP   10.100.0.1    <none>       443/TCP          24h
my-service          NodePort    10.100.76.202 <none>       80:31000/TCP     22h
mymail-service      NodePort    10.100.3.14   <none>       80:31001/TCP     11s
```

```
C:\Users\Vimal Daga\Desktop\kube_code>kubectl describe svc mymail-service
Name:                 mymail-service
Namespace:            default
Labels:               app=mymail-service
Annotations:          <none>
Selector:             app=mymail
Type:                 NodePort
IP Family Policy:     SingleStack
IP Families:          IPv4
IP:                   10.100.3.14
IPs:                  10.100.3.14
Port:                 <unset> 80/TCP
TargetPort:           80/TCP
NodePort:             <unset> 31001/TCP
Endpoints:            192.168.5.45:80,192.168.61.170:80,192.168.88.218:80
Session Affinity:     None
External Traffic Policy: Cluster
Events:               <none>
```

- Launch deployment for mysearch.

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: search
spec:
  replicas: 3
  selector:
    matchLabels:
      app: mysearch
    matchExpressions:
      - { key: app, operator: In, values: [mysearch] }
  template:
    metadata:
      name: mysearch-pod
      labels:
        app: mysearch
    spec:
      containers:
        - name: myweb-con
          image: httpd
```

- Launch service for mysearch deployment.

```
apiVersion: v1
kind: Service
metadata:
  name: mysearch-service
  labels:
    app: mysearch-service
spec:
  type: NodePort
  selector:
    app: mysearch
  ports:
    - port: 80
      targetPort: 80
      nodePort: 31002
```

[EKS]

```
C:\Users\Vimal Daga\Desktop\kube_code>notepad deploy.yml

C:\Users\Vimal Daga\Desktop\kube_code>kubectl apply -f deploy.yml
deployment.apps/search created

C:\Users\Vimal Daga\Desktop\kube_code>notepad service.yml

C:\Users\Vimal Daga\Desktop\kube_code>kubectl apply -f service.yml
service/mysearch-service created
```

```
C:\Users\Vimal Daga\Desktop\kube_code>kubectl describe svc mysearch-service
Name:                mysearch-service
Namespace:           default
Labels:              app=mysearch-service
Annotations:         <none>
Selector:            app=mysearch
Type:               NodePort
IP Family Policy:   SingleStack
IP Families:        IPv4
IP:                 10.100.38.110
IPs:                10.100.38.110
Port:               <unset> 80/TCP
TargetPort:         80/TCP
NodePort:           <unset> 31002/TCP
Endpoints:          192.168.40.110:80,192.168.5.57:80,192.168.80.78:80
Session Affinity:   None
External Traffic Policy: Cluster
Events:             <none>
```

- You can use annotations to specify the type of Application Load Balancer (ALB) that you want to use.

```
kind: Ingress
metadata:
  name: my-app-ingress
  annotations:
    alb.ingress.kubernetes.io/scheme: internet-facing
    alb.ingress.kubernetes.io/healthcheck-protocol: HTTP
    alb.ingress.kubernetes.io/healthcheck-port: traffic-port
    alb.ingress.kubernetes.io/healthcheck-path: /
    alb.ingress.kubernetes.io/healthcheck-interval-seconds: '15'
    alb.ingress.kubernetes.io/healthcheck-timeout-seconds: '5'
    alb.ingress.kubernetes.io/success-codes: '200'
    alb.ingress.kubernetes.io/healthy-threshold-count: '2'
    alb.ingress.kubernetes.io/unhealthy-threshold-count: '2'
spec:
```

- Rules of /mail path: That will go to mymail-service

[EKS]

```
spec:
  ingressClassName: awesome-class
  rules:
  - http:
      paths:
      - path: /mail
        pathType: Prefix
        backend:
          service:
            name: mymail-service
            port:
              number: 80
```

➤ Rules for /search path: That will go to mysearch-service

```
- path: /search
  pathType: Prefix
  backend:
    service:
      name: mysearch-service
      port:
        number: 80
```

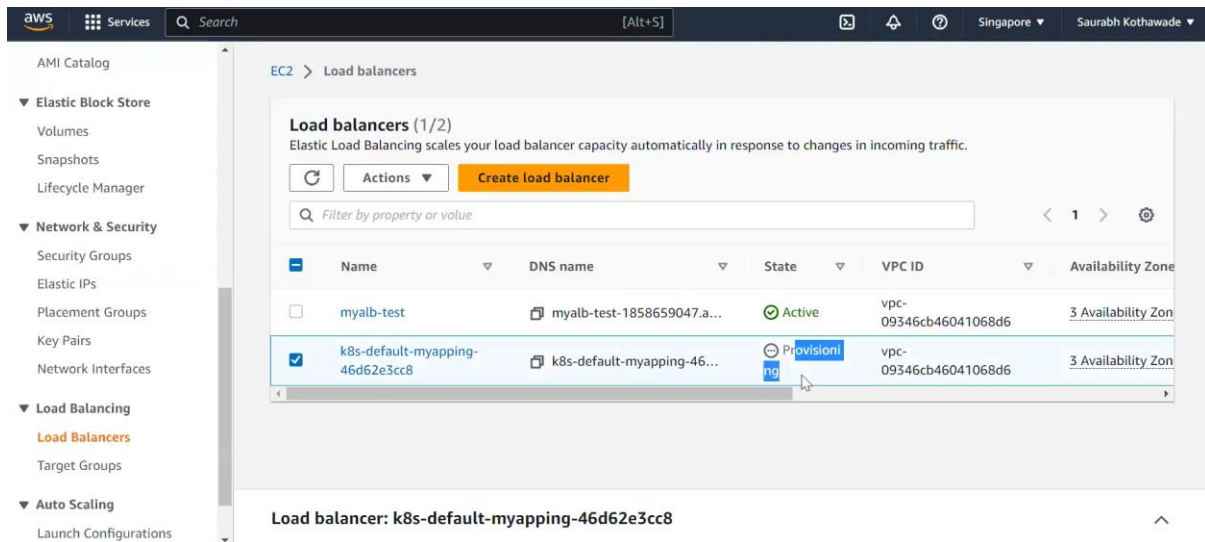
➤ Create an ingress rule.

```
C:\Users\Vimal Daga\Desktop\kube_code>kubectl apply -f ingress-alb-rule.yml
ingress.networking.k8s.io/my-app-ingress created
```

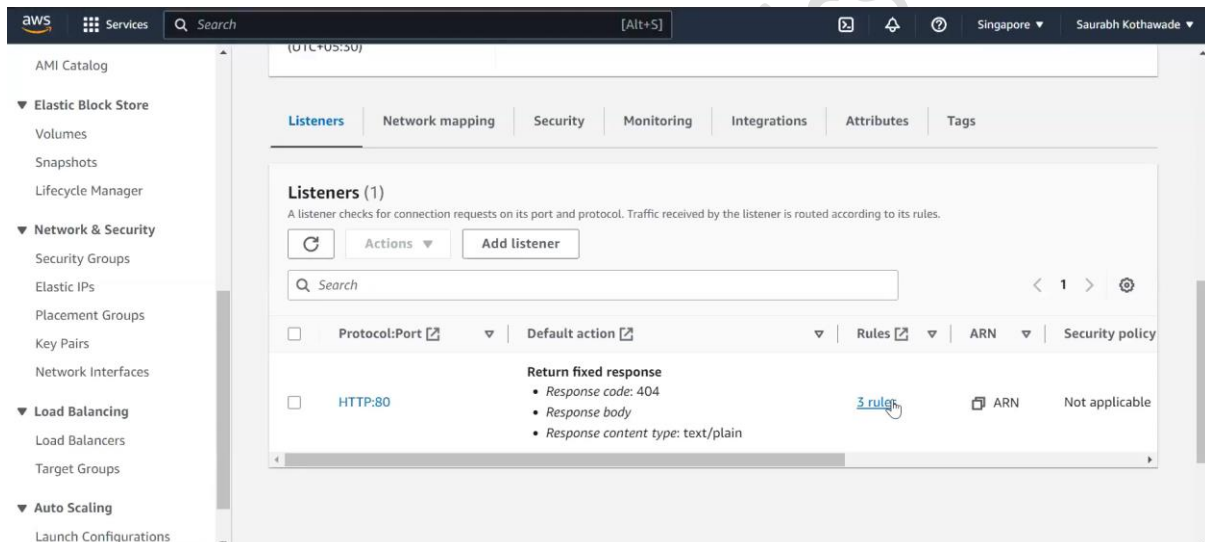
```
C:\Users\Vimal Daga\Desktop\kube_code>kubectl get ingress
NAME                                CLASS      HOSTS                                ADDRESS                                                                 PORTS
minimal-ingress                    awesome-class  *                                myalb-test-1858659047.ap-southeast-1.elb.amazonaws.com                80
my-app-ingress                     awesome-class  *                                k8s-default-myappi-46d62e3cc8-754101094.ap-southeast-1.elb.amazonaws.com 80
```

➤ After that, we can see that a load balancer will be immediately generated if we check the AWS console.

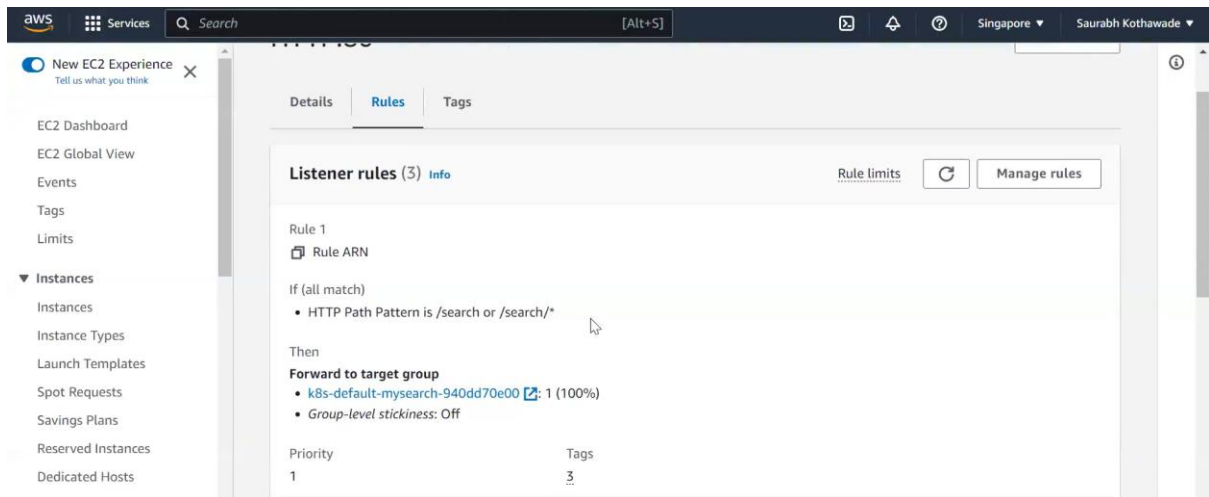
[EKS]



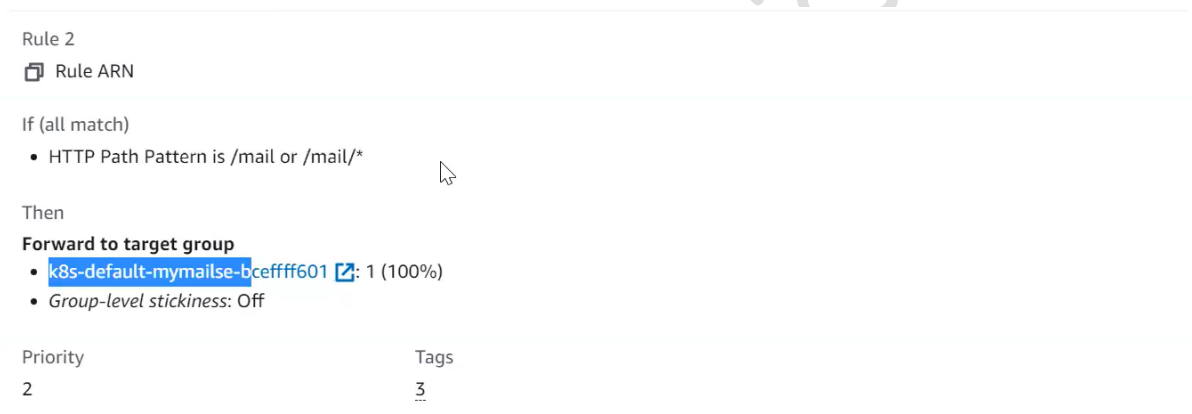
➤ In the AWS console, we can view the rules.



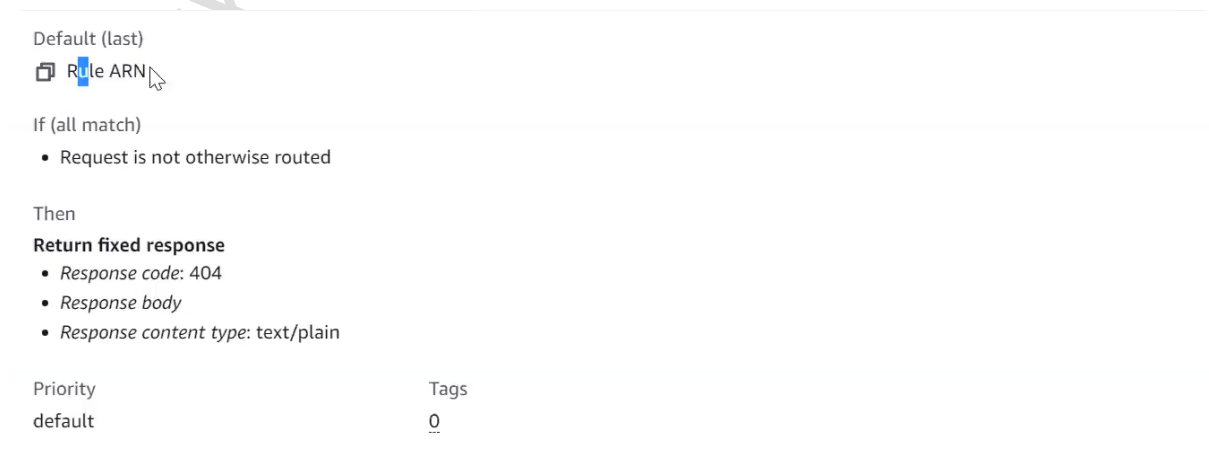
➤ Rule 1: For “/search”



➤ Rule 2: For “/mail”



➤ We will receive a 404 response code kind of page if we enter the incorrect path.



- When you define an Ingress rule in Kubernetes, the path that you specify must match a valid URL path that can be handled by your application. For example, the "search" folder should be present in the web server's home directory if the path is "/search".

```
C:\Users\Vimal Daga\Desktop\kube_code>kubectl exec -it mail-6f5676c969-tmcwq -- bash
[root@mail-6f5676c969-tmcwq /]# cd /var/www/html/
[root@mail-6f5676c969-tmcwq html]# mkdir mail
[root@mail-6f5676c969-tmcwq html]# cd mail/
[root@mail-6f5676c969-tmcwq mail]# cat > index.html
i m mail 3
[root@mail-6f5676c969-tmcwq mail]# exit
exit
```

```
C:\Users\Vimal Daga\Desktop\kube_code>curl http://k8s-default-myapping-46d62e3cc8-754101094.ap-southeast-1.elb.amazonaws.com/mail/index.html
i m mail 3

C:\Users\Vimal Daga\Desktop\kube_code>curl http://k8s-default-myapping-46d62e3cc8-754101094.ap-southeast-1.elb.amazonaws.com/mail/index.html
i m mail 2

C:\Users\Vimal Daga\Desktop\kube_code>curl http://k8s-default-myapping-46d62e3cc8-754101094.ap-southeast-1.elb.amazonaws.com/mail/index.html
i m mail 3
```

- Same applies for each single pod/container.

```
C:\Users\Vimal Daga\Desktop\kube_code>kubectl exec -it search-79c45fbdc4-6n7tl -- bash
root@search-79c45fbdc4-6n7tl:/usr/local/apache2# pwd
/usr/local/apache2
root@search-79c45fbdc4-6n7tl:/usr/local/apache2# ls
bin build cgi-bin conf error htdocs icons include logs modules
root@search-79c45fbdc4-6n7tl:/usr/local/apache2# cd htdocs/
root@search-79c45fbdc4-6n7tl:/usr/local/apache2/htdocs# ls
index.html
root@search-79c45fbdc4-6n7tl:/usr/local/apache2/htdocs# mkdir search
root@search-79c45fbdc4-6n7tl:/usr/local/apache2/htdocs# cd search/
root@search-79c45fbdc4-6n7tl:/usr/local/apache2/htdocs/search# cat > index.html
i m search
root@search-79c45fbdc4-6n7tl:/usr/local/apache2/htdocs/search# exit
exit
```

- **Path types:**
 - 1) Exact: Matches the URL path exactly and with case sensitivity.
 - 2) Prefix: Matches based on a URL path prefix split by /

Examples

Kind	Path(s)	Request path(s)	Matches?
Prefix	/	(all paths)	Yes
Exact	/foo	/foo	Yes
Exact	/foo	/bar	No
Exact	/foo	/foo/	No
Exact	/foo/	/foo	No
Prefix	/foo	/foo , /foo/	Yes
Prefix	/foo/	/foo , /foo/	Yes
Prefix	/aaa/bb	/aaa/bbb	No
Prefix	/aaa/bbb	/aaa/bbb	Yes
Prefix	/aaa/bbb/	/aaa/bbb	Yes, ignores trailing slash
Prefix	/aaa/bbb	/aaa/bbb/	Yes, matches trailing slash

- **Host-based Ingress** rules in Kubernetes are used to route incoming requests based on the host name specified in the request's HTTP header.

```

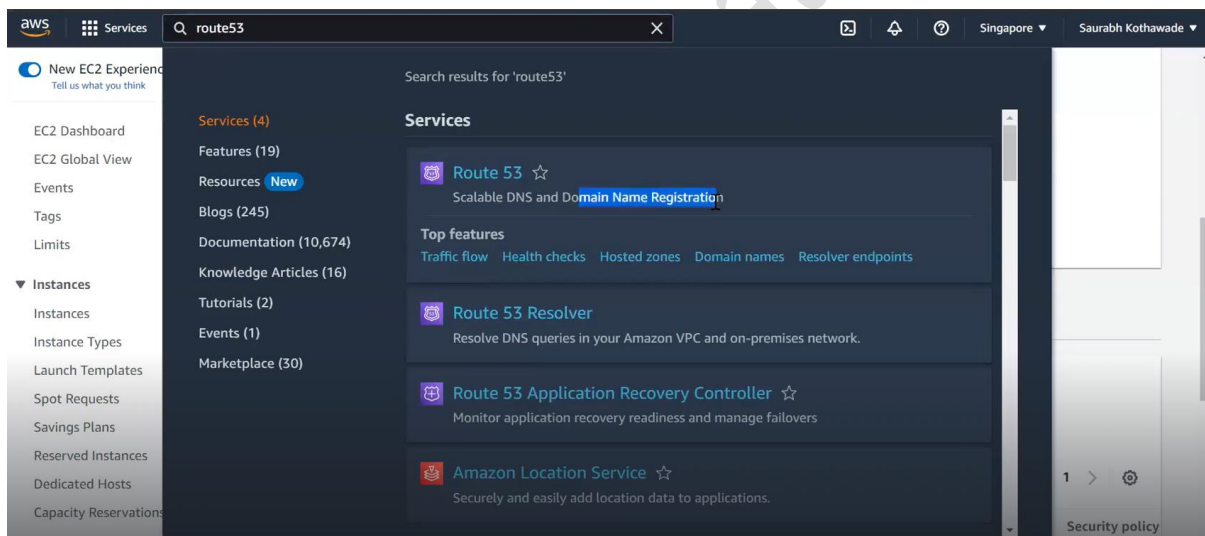
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: ingress-wildcard-host
spec:
  rules:
  - host: "foo.bar.com"
    http:
      paths:
      - pathType: Prefix
        path: "/bar"
        backend:
          service:
            name: service1
            port:
              number: 80
  - host: "*.foo.com"
    http:
      paths:
      - pathType: Prefix
        path: "/foo"
        backend:
          service:
            name: service2
            port:
              number: 80

```

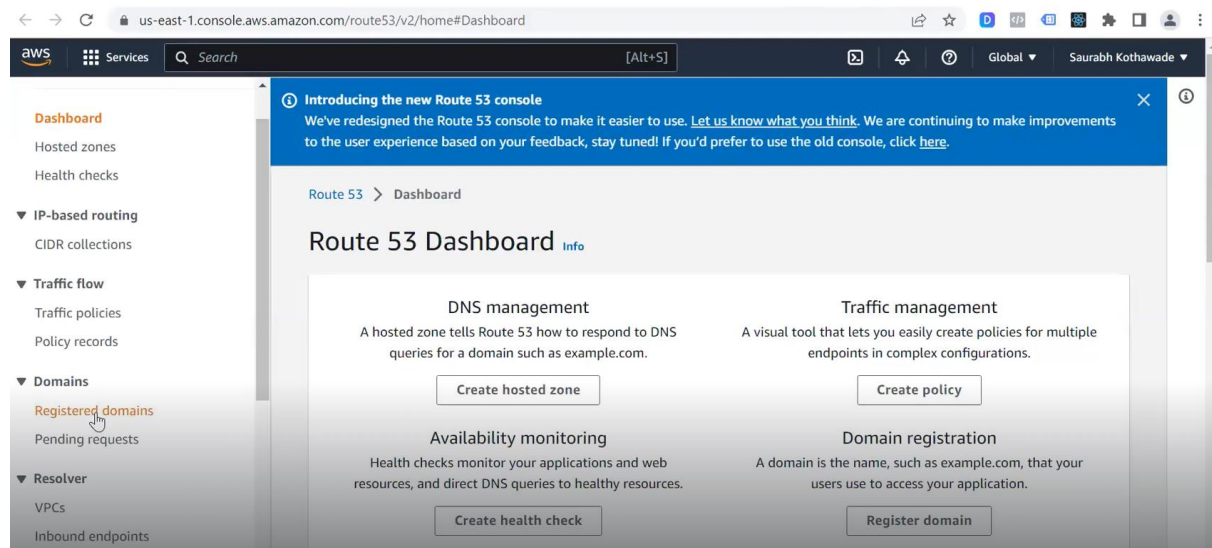
- In Kubernetes, Ingress rules are processed in order, and the first rule that matches the incoming request is used to route the traffic. This means that

if you define a catch-all rule with a path of / at the beginning of your Ingress rule list, it will match all incoming requests, even if there are more specific rules defined later in the list.

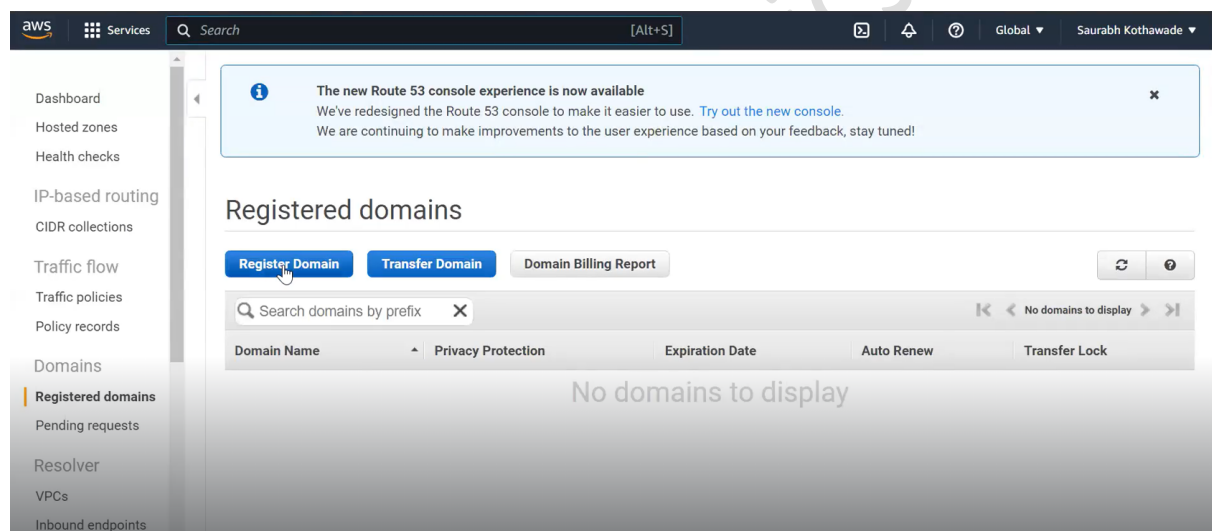
- To avoid this issue, you should avoid defining a catch-all rule with a path of / at the beginning of your rule list. Instead, you can define a catch-all rule at the end of your list of rules.
- **To convert an HTTP endpoint to HTTPS in an Amazon Elastic Kubernetes Service (EKS) cluster:**
- Before setting up HTTPS, you need to have a domain name for your service and an SSL certificate that matches the domain name:
- Register a domain name:



- Click on the "Registered Domains" option in the left-hand menu.



- Click on the "Register Domain" button.



- Enter the domain name you want to register in the "Domain name" field.
- If the domain name is available, select it and click on the "Add to cart" button.

The screenshot shows the AWS Route 53 console. The search bar contains 'lwindia.com'. The results show that 'lwindia.com' is unavailable. Below this, a table of related domain suggestions is displayed. The table has columns for Domain Name, Status, Price /1 Year, and Action. The domains listed are: helwindia.com, jzindia.com, jzindia.net, lwindia.mobi, lwindia.net, lwindia.ninja, and lwindia.org. The prices range from \$11.00 to \$30.00. The 'jzindia.net' domain is marked as 'Available - In Cart'. On the right side, there is a summary for 'jzindia.net' showing a registration price of \$11.00 for 1 year, a subtotal of \$11.00, and a section for 'Monthly Fees for DNS Management'.

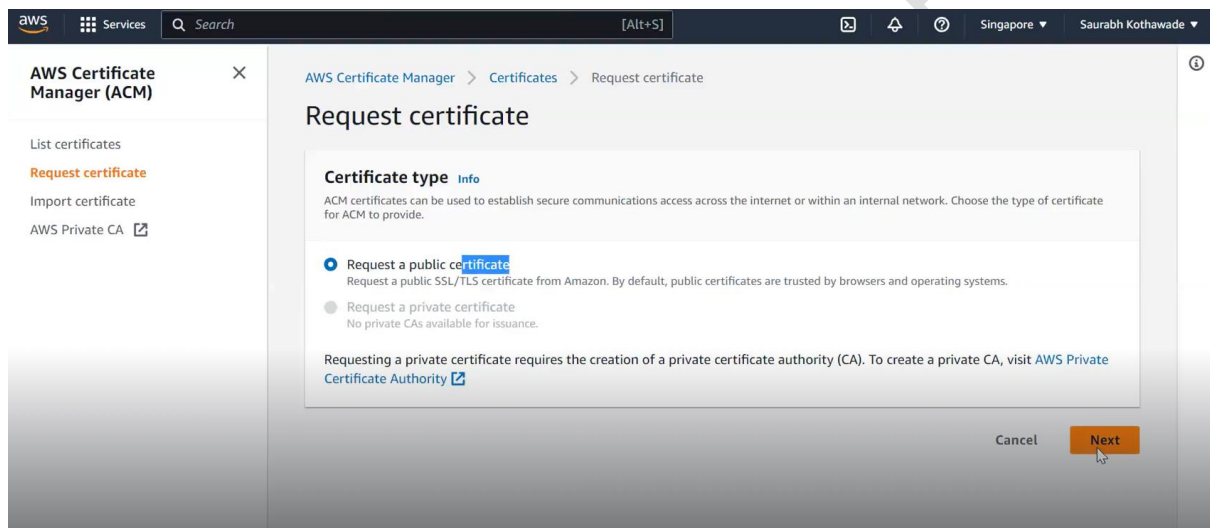
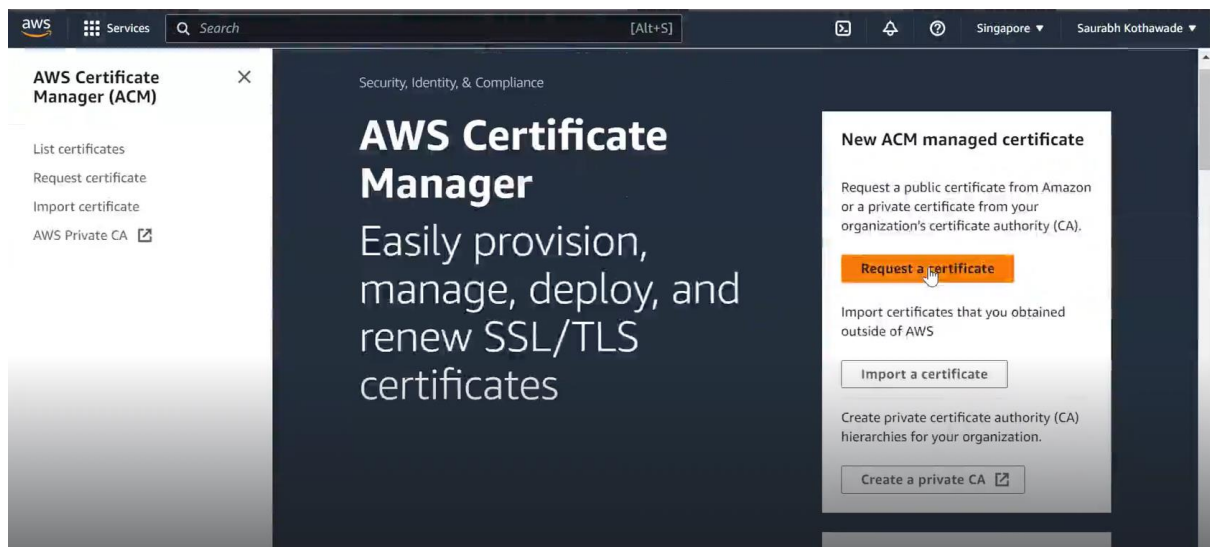
Domain Name	Status	Price /1 Year	Action
lwindia.com	Unavailable		
helwindia.com	Available	\$13.00	Add to cart
jzindia.com	Available	\$13.00	Add to cart
jzindia.net	Available - In Cart	\$11.00	Add to cart
lwindia.mobi	Available	\$30.00	Add to cart
lwindia.net	Available	\$11.00	Add to cart
lwindia.ninja	Available	\$18.00	Add to cart
lwindia.org	Available	\$12.00	Add to cart

- Create an SSL certificate:

The screenshot shows the AWS IAM console. The search bar contains 'certi'. The results show a list of services and features. The services listed are: Certificate Manager, AWS Private Certificate Authority, and AWS IQ. The features listed are: Audit and IoT Core feature. The left sidebar shows the navigation menu with options like Dashboard, Hosted zones, Health checks, IP-based routing, CIDR collections, Traffic flow, Traffic policies, Policy records, Domains, Registered domains, Pending requests, Resolver, VPCs, Inbound endpoints, and Outbound endpoints.

- Click on the "Request a certificate" button.

[EKS]



- Enter the domain name you want to secure with SSL and click on the "Next" button.

Request public certificate

Domain names
Provide one or more domain names for your certificate.

Fully qualified domain name [Info](#)

*.jzindia.net

[Add another name to this certificate](#)

You can add additional names to this certificate. For example, if you're requesting a certificate for "www.example.com", you might want to add the name "example.com" so that customers can reach your site by either name.

Validation method [Info](#)
Select a method for validating domain ownership.

☒ DNS validation - recommended
Choose this option if you are authorized to modify the DNS configuration for the domains in your certificate request.

- Click on the "Request" button to submit your certificate request.

Key algorithm [Info](#)
Select an encryption algorithm. Some algorithms may not be supported by all AWS services.

☒ RSA 2048
RSA is the most widely used key type.

☐ ECDSA P 256
Equivalent in cryptographic strength to RSA 3072.

☐ ECDSA P 384
Equivalent in cryptographic strength to RSA 7680.

Tags [Info](#)
To help you manage your certificates, you can optionally assign your own metadata to each resource in the form of tags.

No tags associated with this resource.

[Add tag](#)

You can add 50 more tag(s).

Cancel Previous Request

- Then after that create a record set in the "Route53" service.
- After completing the above setup you can specify that all traffic should be redirected to HTTPS.
- **alb.ingress.kubernetes.io/listen-ports** specifies the ports that ALB used to listen on.
- `alb.ingress.kubernetes.io/listen-ports: '[{"HTTP": 80}, {"HTTPS": 443}, {"HTTP": 8080}, {"HTTPS": 8443}]'`

➤ ***alb.ingress.kubernetes.io/ssl-redirect*** enables SSLRedirect and specifies the SSL port that redirects to.

- `alb.ingress.kubernetes.io/ssl-redirect: '443'`
- ***alb.ingress.kubernetes.io/certificate-arn*** specifies the ARN of one or more certificate managed by AWS Certificate Manager
- `alb.ingress.kubernetes.io/certificate-arn: arn:aws:acm:us-west-2:xxxxxx:certificate/xxxxxxx`